

DESIGN AND DEVELOPMENT OF A LOW- COST ARDUINO FOUR POINT PROBE

Abstract

This paper details the design, development, components and code to build an affordable and working four-point probe. It contains all the equations needed to find resistance, resistivity and conductivity alongside geometry equations for the materials and the list of all components needed for the probe. This paper also contains the findings from the experiments performed showcasing the strengths and weaknesses found in the probe and what can be improved upon and what new methods could be incorporated when taken further. These findings along with the guide show how to build a functioning four-point probe.

Lewis Baker

32111512@student.uwl.ac.uk

Contents

Introduction	2
Methodology	2
2.1 Equations	2
2.2 Materials and their uses.....	3
2.3 KICAD schematic	5
Experiment and Results.....	7
3.1 Probe readings predictions	7
Discussion	9
4.1 Problems found	9
Conclusion.....	10
References	10
Abstract	11
Arduino IDE Code	11
Python Code	21
Figure 1: KICAD schematic of four-point probe	5
Figure 2: Image of four-point probe	7
Figure 3: table of measurements for materials.....	8
Figure 4: Table of data obtained from probe.....	8

Introduction

The four point probe method, sometimes referred to as the Kelvin method, was created in 1915 by Frank Wenner as he was trying to accurately measure the resistance of the planet Earth. His proposal was that off a four-point (going forward referred as 4P) geometry and this would minimize issues caused by both wiring and contact of probe to material. 39 years later in 1954 this method would be used to measure the resistivity (ρ) of semiconductor wafer materials [1].

Now the four point probe method is a crucial tool in the engineering and material science world, used in every laboratory to measure the resistance (Ω), resistivity and conductivity (σ). It can be taken a step further with the Van der Pauw method and hall method enabling the user to study the measurement of carrier behavior and doping within a material.

This report details a basic guide on how to build your own 4P probe with basic electronics kits and code to be able to effectively measure the resistance of any conducting material available of lengths and thickness. The equations for sheet resistance and various other geometry factors for creating the material sample plus the full code designed to showcase the findings effectively will be shown with detailed analysis showcasing how this can be rebuilt and improved upon.

Methodology

2.1 Equations

There are a number of different equations and geometric factors needed to be used in order to produce an accurate and working probe these are, but not limited to: sheet resistance, circular samples, rectangular samples, sample thickness and the known equations used in electrical work such as ohms law, resistivity and conductivity. The equations used in this paper are shown below.[2]

Table 1: list of equations used for probe

Sheet Resistance	$Rs = \frac{\pi}{\ln 2} * \frac{\Delta V}{I}$
------------------	---

Ohms law	$R = \frac{V}{i}$
Resistivity	$\rho = R * \frac{A}{L}$
Conductivity	$\sigma = \frac{1}{\rho}$
Cross sectional area	In code
Percent deviation formula	$\frac{\text{experimental} - \text{theoretical}}{\text{theoretical}} * 100$

2.2 Materials and their uses

As the whole point of this paper was to create an efficient and low-cost 4P probe, the components and materials needed to be easily found and cost as little as possible. Most of the items needed are basic electronics such as a breadboard, resistors, jumper wires and buttons. But others will need to purchase separately. A table of items and the reasons they are needed for the probe is shown below. These items were chosen for their availability, low cost and their efficiency.

Table 2: List of components and their uses

Breadboard	To insert and power all the components
Arduino Uno (any type of Arduino is fine)	The microcontroller that is the heart of the probe, can send and receive data for ADC.
Jumper wires	To send electricity across the board.
LM317	To provide a constant current source to the first probe
ADS1115	A powerful ADC used to send much finer readings to Arduino which will then be sent to the laptop
LCD I2C	To display the current stages in the probes use
Resistors	To reduce the voltage flow so everything works accordingly
Thermistor	To read the temperature of the space when the probe is reading the material
Pogo pins	To be pressed against the material to inject current and to receive the voltage drop.
Buttons	To press to start each reading and to Reset the probe
Solder and Soldering Iron	To use to solder the pogo pins onto the ends of the jumper wires

An Arduino Uno board is used as it is a beginner friendly board and as the microcontroller it can read all the measurements from the ADS1115 using the I^2C and send them to the computer through the USB, it can perform the calculations given to it from the Arduino code for resistance, resistivity and conductivity and send it the Python through the USB. The ADS1115 is used as it has up to 16-bit resolution (65536) for ADC (analog to digital conversion) whereas the Arduino Uno only has 10 bits (1024). So, by using the ADS it can read much more finely which can then be used to calculate accurate resistance readings than that of Arduino alone. This is then sent back to the Arduino using the SDA (Serial Data line) and the SCL (serial clock line). The LM317 is used as it can provide both a constant voltage and current source. The LM317 is an adjustable positive voltage regulator, which has a voltage reference of 1.25V and can be programmed using two external resistors. For the 4P probe a constant current source is needed as a wire is connected straight from the adjust leg with a pogo pin soldered onto the opposite end, and this will be the input probe wire. A resistor with a rating of 220Ω was used between the legs of the adjust and output to get a current of $I = \frac{V}{R}$ which would have been $I = \frac{1.25}{220} = 0.00568A$.

A thermistor was incorporated as an extra sensor to the probe to record the surrounding temperature to add as an extra variable to the measurements as heat can directly affect the resistance of a material. Temperature was not something that could be controlled for the building of this probe, just an extra piece of information.

The LCD I2C is used as it is compatible with the Arduino as they both use the serial bus pins (A4 & A5) which can be used to send and receive text and information to showcase on the display such as “Reset” and “Test 1: complete”.

Two buttons are used in the making of the 4P point probe. One button is used to start the probe and to press when the user wants to take the next test after placing the probe pins onto the material and the other button is a hard reset which acts as an interrupt, when pressed the entire data and information is wiped clean and the probe reverts back to its beginning point to start the process over again.

Spring loaded pogo pins act as the bridge between the probe and the material, they are what is pressed to the material and to both inject the current and receive the voltage. By using spring loaded ones they provide a repeatable known contact force when applied to the material and the fineness of the tip provides a well defined and repeatable contact point for accurate resistance measurements. [\[3\]](#)

The materials used were chosen because of their ability to conduct and their shape. The three materials are a steel nail, the lead of a pencil and the aluminum lid of a can. Both the pencil lead and the nail had a length of 57mm or 0.057m or 5.7cm. The spacing between the probe pins was 10mm. The radius of both the nail and the length was found by halving the diameter, which itself was found by holding a basic ruler to the items. The radius of the nail was 0.002m and the lead was 0.001m.

2.3 KICAD schematic

[illegible]

2.4 code behind the probe

To make sufficient and working probe two sets of code must be programmed into the Arduino. First the code that performs the calculations to find the resistance, resistivity and conductivity, this can be done in Arduino IDE. Next is the code that takes all the information is presents it in graphs and table, this can be done in Python and Pandas. The code written in the IDE is what is uploaded to the Arduino Uno, and this allows for all the tests to be done and uploaded back to the computer and then to the python through serial data.

Arrays were used in the IDE to hold the values of Radius, width and thickness for the calculations of resistance, conductivity and resistivity. They were also used to hold the different gains and the conversion factor. The ADS1115 has a PGA (programmable gain amplifier) and the higher the gain the smaller the voltage ranges which means the ADC has better use for measuring smaller voltages. I.E. gain of 16 = $\pm 0.256V$ with the LSB (least significant bit) being 0.0000078125V. Lower gain equals higher voltage range allowing higher voltages to be measured. I.E. gain or 2/3 = $\pm 6.114V$ with LSB being 0.0001875V

The ADS1115 only reads digits from 0-32768 in the positive range so the code must reflect this. The voltage in the calculateVoltage variable has the equation of voltage = adcvalue * conversionfactor[currentgain]. With adcvalue being the reading of two inner probes from the ads1115 which is the voltage multiplied by the conversion factor corresponding to the selected gain.

The conversion factor also means that if the reading received in the ads was a small reading of for example, 9000 while using a lower gain setting, much of the available measurement is being left unused. So, an open-sourced code was found and used that consisted of using if statements that allow the code to adjust the ADS1115 voltage gain based on the ADC reading. This allows higher ADC readings to use lower gain measurements and vice versa for better voltage readings [\[4\]](#).

To calculate resistivity, I needed the dimensions of the materials I was using. The probe spacing was 10mm apart so that was the length and both radius and thickness are discussed above (3.2). If statements were used. As there are for this test only three materials being used, the last one being the only one needing the sheet resistance equation, which is $\rho = R_s * t$. R_s being sheet resistance and t being thickness.

The Arduino IDE code was designed to take the raw inputs from the hardware and turn it into the resistance, resistivity and conductivity from three different tests per each of the three materials and send all of that over the python for the presentation. A send all variable was created that would send all the information via serial print to the python.

The python code was designed purely with presentation in mind. All off the information in

python was sent over from Arduino IDE via the serial print USB line. The Python code received all the information and by using arrays can file it under the correct category such as material, test, resistance, resistivity and conductivity. It is programmed with how many readings it can expect by designated materials = 3 alongside tests = 3 then multiplying them together. All the readings are printed together by appending each bit of information. Alongside this, using python's pandas, the average of each category of reading is found and presented into three distinct and well labeled graphs at the end showcasing resistivity, conductivity and resistance alongside a table laid all the readings including the temperature of the space in the terminal.

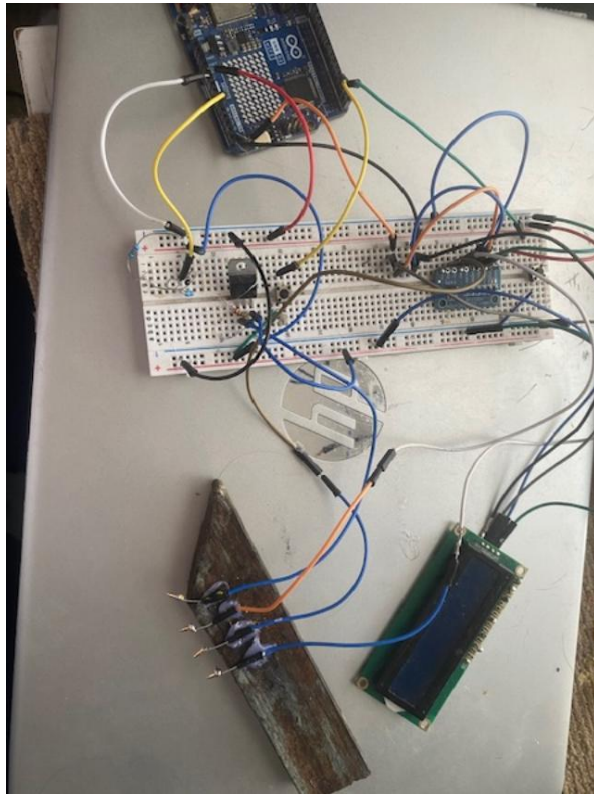


Figure 2: Image of four-point probe

Experiment and Results

3.1 Probe readings predictions

Before performing the tests, it can be a good idea to try to predict what the readings will be by using the parameters. The table for the resistivity and conductivity of the materials used in this paper are shown below. [\[5\]](#)

Materials	Resistivity	Conductivity
Steel nail	$0.72 * 10^{-6}$	$1.39 * 10^6$
Pencil graphite	$3.5 * 10^{-5}$	$2.9 * 10^4$
Aluminum	$2.82 * 10^{-8}$	$3.55 * 10^7$

Figure 3: table of measurements for materials

Taking these and rewriting the equations in the table shown in methodology, the predicated resistance can be found.

3.2 probe reading results

To take the tests, the user must hold the apparatus that holds the wires of the probe and place them thoroughly on the material and hold while the machine does the tests. The table below is generated by python pandas and shows all the data for resistance, resistivity and conductivity and the average of all three tests pr material.

```

DATA:
  material test voltage resistance resistivity conductivity temperature
0      Nail 0 0.000039 0.005661 0.000007 140565.609375 230.38
1      Nail 1 0.000039 0.005661 0.000007 140565.609375 274.24
2      Nail 2 0.000063 0.009058 0.000011 87853.507812 277.49
3 Pencil Lead 0 0.000063 0.009058 0.000003 351414.031250 277.49
4 Pencil Lead 1 0.000063 0.009058 0.000003 351414.031250 274.24
5 Pencil Lead 2 0.000063 0.009058 0.000003 351414.031250 271.16
6 Aluminum Can 0 0.000063 0.009058 0.000036 27869.759766 274.24
7 Aluminum Can 1 0.000039 0.005661 0.000022 44591.613281 277.49
8 Aluminum Can 2 0.000039 0.005661 0.000022 44591.613281 277.49
  material avg_resistance avg_resistivity avg_conductivity
0      Nail 0.006793 0.000009 122994.908854
1 Pencil Lead 0.009058 0.000003 351414.031250
2 Aluminum Can 0.006793 0.000027 39017.662109

```

Figure 4: Table of data obtained from probe

By swapping around the equations a prediction for what resistance could have been versus what was found, a table showcasing this is shown below, based off averages and the percentage difference can also be found.

Material	Predicted	Real result	% difference
Nial	0.000573	0.006793	1085%
Pencil graphite	0.1114	0.009058	-92%
Aluminum	0.0000322	0.006793	20996%

By using the percentage deviation formula, the % difference for each material can be found.

The graphs below showcasing resistance, resistivity and conductivity.

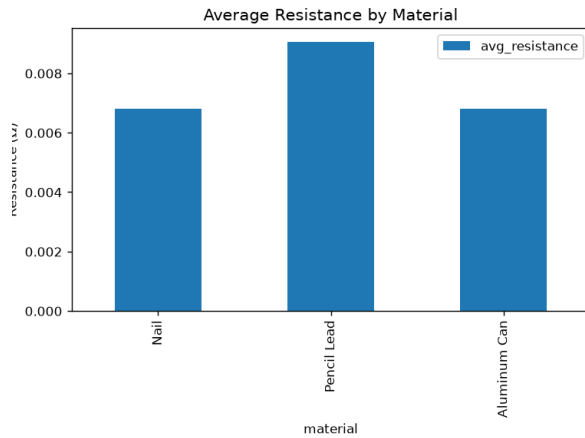


Figure 5: Graph showcasing average resistance

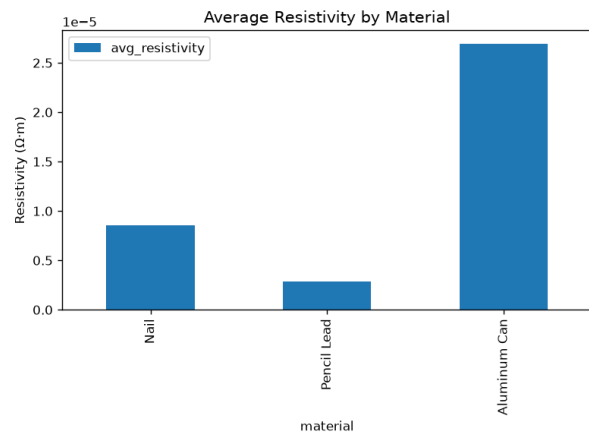


Figure 6: Graph showcasing average resistivity

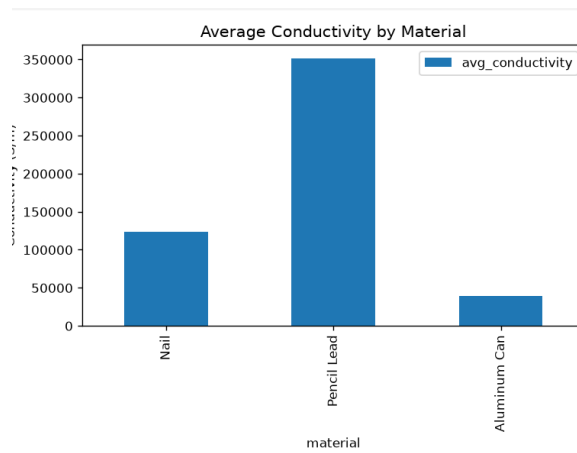


Figure 7: Graph showcasing average conductivity

Discussion

4.1 Problems found

When presented with the findings and comparing them with the predicted results, several discrepancies arise. First, the percentage difference between the predicted and actual result is substantially different and changes from each material, not one material was close to the predicted reading to get. This can be due to a few reasons such as the creation of the probe itself. If the pogo pins were not perfectly 10mm apart in length even a change in 1-2mm can affect the resistance readings. If the apparatus that was pressing the pogo pins to the material shifted during tests, was not firm enough or if possibly one of the pogo pins slipped off the material during the tests and was not noted. This can affect the reading. Mechanical clamping would be preferable than handheld.

In the above graphs and tables, it is noted that pencil lead should be pencil graphite. The tests taken were done using the lead resistivity of 0.22×10^{-6} , this oversight produced incorrect readings, and the equations were rewritten with the correct material in mind, but the labels were not updated.

For Aluminum there are also a few different reasons as to why average resistance is the same as the steel nail. One reason could be oxide layer formation Al_2O_3 . As opposed to graphite and steel, aluminum forms a thin electrically insulating layer upon contact with air. This could prevent accurate reading as well as producing one that is remarkably far from the theoretical one.

Conclusion

The average resistance, resistivity and conductivity of three different materials were found from a cheap, easy to design and create four-point probe and were organized into graphs and tables for the reader to note based off open-sourced hardware and easily accessible components, showcasing the highest and lowest orders of the three parameters. Faults were also found in the probe which can be rectified, removed and updated when chosen to be remade and improved.

Compared to theoretical predictions and literature showed a number of discrepancies which can easily be explained by several reasons, mainly down to the construction of the probe. This showcases that while the probe does its job, it can be held back by mechanical fixture and lack of precision equipment. With these improvements the design has a clear chance of providing results much closer to theoretical results and can be improved with new techniques such as Hall method or Van der Pauw.

References

- [1] Wenner, F. (1915). A method for measuring Earth resistivity. Journal of the Washington Academy of Sciences, 5(16), 561–563. <http://www.jstor.org/stable/24520849>
- [2] Ossila, “Calculate Sheet Resistance Using the Four-Probe Method”, Ossila enabling science, Available <https://www.ossila.com/pages/sheet-resistance-theory>
- [3] Autodesk, “Making a 4 Probes Resistance Measurement Setup” Autodesk Instructables, Available <https://www.instructables.com/Making-a-4-Probes-Resistance-Measurement-Setup/>

[4] Github, “ADS1115- Auto gain”, Github AKStudios, Available,
<https://github.com/AKstudios/ADS1115-auto-gain/blob/master/ADS1115-auto-gain.ino>

[5] P Scherz & S Monk. Practical Electronics for Inventors, 4th Edition, McGraw-Hill Education, 2016

Abstract

Arduino IDE Code

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#define MATERIALS 3

#define TESTS 3

#include <Adafruit_ADS1X15.h>

const int thermistor = A0;

const byte buttonpin = 2;

const byte resetpin = 3;

const float current = 0.0056;

int materials = 0;

int tests = 0;

LiquidCrystal_I2C lcd(0x27,16,2);

Adafruit_ADS1X15 ads;

byte currentgain = 5;

volatile bool resetFlag = false;

float length[2] = {0.010, 0.010}; // nail and pencil lead

float radius[2] = {0.002, 0.001}; // nail and penicl lead

float thickness[1] = {0.000874}; // aluminum can

adsGain_t gainplace[6] {
```

```
GAIN_TWOTHIRDS,  
GAIN_ONE,  
GAIN_TWO,  
GAIN_FOUR,  
GAIN_EIGHT,  
GAIN_SIXTEEN,  
};
```

```
float conversionfactor[6] {  
    0.0001875,  
    0.000125,  
    0.0000625,  
    0.00003125,  
    0.000015625,  
    0.0000078125,  
};
```

```
float resistanceValue[MATERIALS][TESTS];  
float voltageValue[MATERIALS][TESTS];  
void sendall();  
void resetISR();  
void performreset();  
void buttonWait();  
void adjustGain();  
int16_t readADC();  
float calculateVoltage(int16_t adc);
```

```
float calculateResistance(float voltage);  
float calculateResistivity(float resistance, float voltage, float current, int materials);  
float calculateConductivity(float resistivityValue);  
float temperature();  
void displayLCD();
```

```
void setup() {  
    pinMode(buttonpin, INPUT_PULLUP);  
    pinMode(resetpin, INPUT_PULLUP);  
    Serial.begin(9600);  
    Wire.begin();  
    lcd.init();  
    lcd.backlight();  
    lcd.setCursor(0,0);  
    lcd.print("probe");  
    lcd.setCursor(0,1);  
    lcd.print("is ready");  
    if (!ads.begin(0x48)) {  
        Serial.println("ads1115 not found");  
        lcd.clear();  
        lcd.print("ads error");  
        while(1);  
    }  
    Serial.println("ads1115 found");  
    ads.setGain(gainplace[currentgain]);  
    delay(2000);
```

```

    attachInterrupt(digitalPinToInterrupt(3), resetISR, FALLING);
}

void resetISR() {
    resetFlag = true;
}

void performreset(){
    for (int m = 0; m < MATERIALS; m++) {
        for (int t = 0; t < TESTS; t++) {
            resistanceValue[m][t] = 0.0;
        }
    }

    materials = 0;
    tests = 0;
    currentgain = 5;
    ads.setGain(gainplace[currentgain]);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("reset!!!");
    delay(1500);
    Serial.println("reset!!!");
    lcd.setCursor(0,0);
    lcd.print("probe");
    lcd.setCursor(0,1);
    lcd.print("is ready");
}

bool buttonwait() {

```

```

while(digitalRead(buttonpin) == HIGH)
{
    if (resetFlag) {
        return false;
    }
    delay(200);
}
delay(500);
return true;
}

float adjustGain(int16_t adcReading) {

    if (adcReading >= 28000 && currentgain > 0) {
        Serial.print("reducing gain");
        currentgain--;
        ads.setGain(gainplace[currentgain]);
    } else if (adcReading <= 8000 && currentgain < 5) {
        Serial.print("increasing gain");
        currentgain++;
        ads.setGain(gainplace[currentgain]);
    }
    return adcReading;
}

```

```

int16_t readADC() {

```

```

int16_t adcValue;

adcValue = ads.readADC_Differential_0_1();

Serial.println("adc read complete");

return adcValue;

}

```

```

float calculateVoltage(int16_t adcValue){

float voltage;

voltage = adcValue * conversionfactor[currentgain];

return fabs(voltage);

}

```

```

float calculateResistance(float voltage) {

float resistance;

resistance = fabs(voltage) / current;

return resistance;

}

```

```

float calculateResistivity(float voltage, float resistance, float current, int materials) {

float resistivityValue;

if (materials < 2) {

float area = PI * radius[materials] * radius[materials];

resistivityValue = (resistance * area) / length[materials];

} else {

float sheet_resistance = (PI / log(2) * (voltage / current));

resistivityValue = sheet_resistance * thickness[materials - 2];

}
}

```

```

    }
    return resistivityValue;
}

float calculateConductivity(float resistivityValue) {
    float conductivityValue = 1.0 / resistivityValue;
    return conductivityValue;
}

float temperature() {
    int reading = analogRead(thermistor);
    float temperature = 1.0 / (
        (1/298.15) +
        (1/3950.0) *
        log(reading / 10000.0)
    ) - 273.15;
    return temperature;
}

void displaylcd() {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("press buttton");
    lcd.setCursor(0,1);
    lcd.print("to measure");
}

```

```

void readinglcd() {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("testing");
}

void finishedlcd(){
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("finished");
}

void sendall() {
    for (int materials = 0; materials < MATERIALS; materials++) {
        for (int tests = 0; tests < TESTS; tests++) {
            float voltage = voltageValue[materials][tests];
            float resistance = resistanceValue[materials][tests];
            float resistivity = calculateResistivity(voltage, resistance, current, materials);
            float conductivity = calculateConductivity(resistivity);
            float temp = temperature();
            Serial.print(materials);
            Serial.print(",");
            Serial.print(tests);
            Serial.print(",");
            Serial.print(voltage, 12);
            Serial.print(",");
            Serial.print(resistance, 12);
            Serial.print(",");

```

```

    Serial.print(resistivity, 12);

    Serial.print(",");

    Serial.print(conductivity, 12);

    Serial.print(",");

    Serial.println(temp);
}
}

Serial.println("DONE");
}

void loop() {
    if (resetFlag) {
        resetFlag = false;
        performreset();
        return;
    }

    for (materials = 0; materials < MATERIALS; materials++) {
        for(tests = 0; tests < TESTS; tests++){

displaylcd();

if (!buttonwait()) {
    resetFlag = false;
    performreset();
    return;
}
readinglcd();
delay(500);

```

```

int16_t adcValue = readADC();
adjustGain(adcValue);
adcValue = readADC();
float voltage = calculateVoltage(adcValue);
voltageValue[materials][tests] = voltage;
resistanceValue[materials][tests] = calculateResistance(voltage);

lcd.clear();
lcd.setCursor(0,0);
lcd.print("Test ");
lcd.print(tests + 1);
lcd.print(" complete");
lcd.setCursor(0,1);
lcd.print("Next test");
delay(1000);
}
}

finishedlcd();
if (!buttonwait()) {

resetFlag = false;
performreset();

return;
}

delay(1000);

```

```
sendall();  
while (!resetFlag) {  
    delay(100);  
}  
}
```

Python Code

```
import serial  
import time  
import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd  
  
arduinoSerial = serial.Serial('COM9', 9600, timeout=1)  
materials = ["Nail", "Pencil Lead", "Aluminum Can"]  
tests_per_material = 3  
expected_readings = len(materials) * tests_per_material  
readings = []  
  
while True:  
    line = arduinoSerial.readline().decode('utf-8').rstrip()  
    if not line:  
        continue  
    print("no data received")  
    if line == "DONE":  
        break  
    data = line.split(",")  
    if len(data) != 7:  
        print("arduino:", line)
```

continue

material = materials[int(data[0])]

test = int(data[1])

voltage = float(data[2])

resistance = float(data[3])

resistivity = float(data[4])

conductivity = float(data[5])

temperature = float(data[6])

reading = {

 "material": material,

 "test": test,

 "voltage": voltage,

 "resistance": resistance,

 "resistivity": resistivity,

 "conductivity": conductivity,

 "temperature": temperature

}

readings.append(reading)

print (f"Reading {len(readings)} / {expected_readings}:"

 f"material {material}, test {test}, voltage {voltage:.2f} V, resistance {resistance:.4f} Ohms,
resistivity {resistivity:.4f} ohm-m, conductivity {conductivity:.4f} s/m, temperature
{temperature:.2f} C")

print("\nALL DATA RECEIVED")

print(f"Total readings: {len(readings)}")

```
df = pd.DataFrame(readings)
print("\nDATA:")
print(df)

comparison = df.groupby("material", sort=False).agg(
    avg_resistance=("resistance", "mean"),
    avg_resistivity=("resistivity", "mean"),
    avg_conductivity=("conductivity", "mean")
).reset_index()

print(comparison)

comparison.plot(kind='bar', x='material', y='avg_resistance', title='Average Resistance by Material')
plt.ylabel("Resistance ( $\Omega$ )")
plt.tight_layout()
plt.show()

comparison.plot(kind='bar', x='material', y='avg_resistivity', title='Average Resistivity by Material')
plt.ylabel("Resistivity ( $\Omega \cdot m$ )")
plt.tight_layout()
plt.show()

comparison.plot(kind='bar', x='material', y='avg_conductivity', title='Average Conductivity by Material')
plt.ylabel("Conductivity (S/m)")
```

```
plt.tight_layout()
```

```
plt.show()
```